

ADR-001: Stack decision

החלטת ארכיטקטורה (ADR)

נוצר על ידי: אלכס • 2026-09-25

תוכן עניינים

Framework: Next.js 15 App Router .2

1. ("כללי החלטה" CLAUDE.md) Decision rule applied

3. Hosting: Netlify (free tier), not Vercel, not Cloudflare Pages

4. Database + Auth + Storage: Supabase (free tier at BUILD/DEV, decision

6. Web push

5. Email: Resend (free tier)

7. Payment-pending expiry job: Netlify Scheduled Function

9. Rule 27 – operations registry: GO

8. Rule 30 – spend caps and alerts (day one)

("כללי החלטה" CLAUDE.md) Decision rule applied 1

1. Platforms include native? No, PWA only. Rule out Expo

2. Self-host / sovereignty requirement? No

3. Sustained realtime/WebSockets? No, web push is fire-and-forget, not a

persistent connection

1. Backend-heavy (queues/workers/ML) bigger than frontend? No, this is a

system. CRUD + one transactional invariant, not a worker-heavy

1. Default applies: **Next.js 15 App Router**

Framework: Next.js 15 App Router 2

Alternative considered: Hono/Express + React-Vite. Pros: no server/client boundary discipline needed, full control of the scheduled job's runtime. Cost: two deployables instead of one, no built-in SSR story, loses next-intl RTL tooling used agency-wide, and this project has no self-host or realtime requirement to justify the extra ops surface for a .single-kitchen app with no dedicated ops person post-launch. Rejected

-Steelman of Next.js for this project: the admin is the single highest risk surface (Yuval sets prices, approves custom cakes, marks orders paid) , `lib/server/`) and Next.js Server Actions + Route Handlers give one place to keep every write path server-only with zero (`import 'server-only'` separate backend process to deploy or monitor, which matters more than usual here because there is no ops team after LAUNCH, only Yuval and an on-call Kivun path. Rule 2 (server/client/API separation) is mandatory anyway given auth + DB writes, so the "structure discipline" cost is paid regardless of framework. Next.js wins on operational simplicity for a one-operator .business, not on technical necessity

Hosting: Netlify (free tier), not Vercel, not Cloudflare Pages 3

, Vercel Hobby explicitly forbids commercial use (verified earlier today , vercel.com/docs/limits/fair-use-guidelines) and this is a commercial bakery .so Hobby is out; Vercel Pro is \$20/month

:Verified today (WebSearch, 2026-09-25)

Netlify free tier: commercial use explicitly allowed (Netlify Support • Forums + [netlify.com blog "Introducing Netlify's Free plan"](https://netlify.com/blog/introducing-netlifs-free-plan): "you can deploy commercial projects... on the Free plan"). Limits: 100GB

bandwidth/mo, 300 build minutes/mo, 125,000 function invocations/mo, 1M edge function invocations/mo. [Netlify pricing guide](#), [Netlify free plan blog post](#)

Cloudflare Pages free tier: commercial use allowed, unlimited static bandwidth, but Pages Functions (needed for SSR/API routes in Next.js) are billed against the Workers free plan: 100,000 requests/day, 10ms CPU per request, 50 subrequests per invocation. Running Next.js App Router on Cloudflare Pages requires `@cloudflare/next-on-pages`, an added adapter layer, and the 10ms CPU ceiling is tight for a server-rendered catalog page hitting Postgres. [Cloudflare Pages pricing docs](#)

Decision: Netlify free tier. It supports Next.js App Router natively the function-invocation ceiling (125k/month, ~4,100/day) (no adapter layer) is far above a single-kitchen bakery's realistic traffic, and commercial use is unambiguous in Netlify's own terms rather than inferred from a CPU-time ceiling. Reassess only if traffic or build-minute usage approaches the free .cap (Rule 30 alert, see Section "Spend cap")

Database + Auth + Storage: Supabase (free tier at BUILD/DEV, **decision**)

4

(below for PROD

Supabase Postgres gives row-level transactional locking (``SELECT ... FOR UPDATE``) needed for the capacity invariant (ADR-002), Supabase Auth covers both admin MFA (TOTP) and optional customer registration with one system instead of two, and Supabase Storage covers both the public catalog-photo bucket and the private custom-cake inspiration-photo bucket with the same RLS model used for the rest of the data. No second vendor needed for three different concerns (DB, auth, storage) is the reason this wins over + composing separate best-of-breed services (e.g., Postgres elsewhere

.Clerk/Auth.js + S3) for a project with no dedicated ops person

:day pause risk (verified today, Supabase docs "Project Pausing")-7

free-tier projects pause after 7 days with no database activity; paid projects (Pro, \$25/mo) are exempt. **Mitigation chosen: the payment-pending

expiry sweep (US-9, runs every 15 minutes via a Netlify Scheduled Function)

writes to the database on every run whether or not it finds an expired

order, which is real, if light, database activity far more often than the

day threshold.** This means the sweep job that MVP already needs for-7

,correctness also solves the pause risk, at no extra cost or complexity

rather than standing up a separate keep-alive ping. If Supabase's own

pause-detection logic requires *user* query volume rather than any write

,the fallback is a second ,(unverified in today's search, sources disagreed)

independent GitHub Actions cron hitting a trivial `SELECT 1` health-check

.route, which costs nothing extra either

"project free-org cap (agency memory: "Free-tier 2-project cap trap-2**

.bit us on Make.com invoice automation before): Rule 6 needs DEV + PROD, i.e

Supabase projects.** Ran's existing Kivun Supabase org already holds 2

projects, so DEV+PROD for this engagement may not fit under Ran's org's free

allotment. **Decision needing Ran/Yuval: create a NEW, separate Supabase

account (and Netlify account) under Yuval's own email/ownership**, not

inside Kivun's existing org. This also correctly places billing and account

ownership with the business that will run this system after the engagement

ends (there is no Kivun contract here, per engagement.md), and sidesteps the

project-cap collision entirely. **Flagged as an open decision for Ran to

**confirm with Yuval before provisioning

Email: Resend (free tier) 5

Verified today (WebSearch): Resend free tier = 3,000 emails/month, capped at 100/day. [Resend pricing](#), [Resend free tier explainer](#)

A single-kitchen bakery's order-confirmation + admin-notification email volume is expected to be far under 100/day; reassess if Phase 2 loyalty .email blasts approach the cap (Rule 30 alert threshold: 80/day sustained)

Web push 6

Standard Web Push API with VAPID keys, no vendor (no Firebase Cloud Messaging needed for a same-origin PWA). \$0, no metered dependency, no Rule .cap needed for this piece specifically 30

Payment-pending expiry job: Netlify Scheduled Function 7

Runs every 15 minutes, executes one idempotent SQL statement per expiry :class (standard 4h, custom-cake 24h)

UPDATE orders SET status='expired' WHERE status='payment_pending' AND ` expires\_at < now()`. This is naturally safe to re-run (a restart or a double-fire updates zero rows the second time), so no separate dedup table is required the way it would be for an `cron_executions`

+ INSERT-based pg_cron job (per the 2026-04-20 learned rule on pg_cron Edge Functions oversell risk) – the WHERE-clause conditional UPDATE is idempotent by construction, not by an added dedup layer. Same job also releases the capacity hold (ADR-002) inside the same transaction as the status flip, so a crash mid-sweep cannot release capacity without also

.marking the order expired, or vice versa

Rule 30 — spend caps and alerts (day one) 8

Every metered account gets a hard cap + an alert to a human, in the unit

:the provider bills in, configured the day the account is created

Netlify: cap at free tier (300 build min/mo, 125k function •

.invocations/mo); alert at 80% via Netlify's own usage notifications

Supabase (once Pro, if ever needed): cap = stay on free unless Rule 30 •

approval for Pro is explicitly taken; alert on approaching the 500MB

.free DB size or 1GB free storage

.Resend: alert at 80/day sustained (before the 100/day hard cap bites) •

.(`infra-spend-caps`) Task included in tasks.json •

Rule 27 — operations registry: GO 9

,Decision: **GO**, scoped to the five business-critical write operations

landed at the same time as the data-model pass (before API routes are

:written), per the rule's own timing requirement

, `declineCustomCakeRequest` , `approveCustomCakeRequest` , `markOrderPaid`

These are exactly the . `updateDayCapacity` , `generateDeliveryList`

operations with real financial/operational consequence and the ones an

agent or future automation (e.g., Phase 2 automated WhatsApp, a future

accounting sync) would need to call safely with RBAC. Read-only catalog

browsing and checkout's own multi-step form are NOT wrapped in the registry

in MVP: the registry is for named business operations with consequence, not

every mutation, and a checkout's own route handler stays the natural place

for that flow. Reassess scope at Phase 2 when Loyalty adds more named

.operations (e.g., `sendLoyaltyBenefit`)

מאושר

מאושר